

Lecture Making Functions to Control Sound in Python

In order to perform experiments, we have to deliver stimuli and collect responses from participants in an experiment.

It makes the most sense to do that on a computer where we can precisely control stimulus presentation, and collect behavioral responses in data files ready for analysis.

There are stimulus, control, and data organization issues that arise in designing and carrying out an experiment that are intimately linked to the design of the experiment.

One size rarely fits all, and for a Psychologist doing research, the ability to program means the ability to implement an experiment precisely as they want to. The same is true for a Neuroscientist.

Nowadays, many (most?) psychological experiments are done online. Even here, you have to program the experiment either in Python or Java Script.

There are also some more general principles of how to organize a program that I will also be introducing.

Why sound?

Most experiments make use of visual stimuli, and in Python there is a module called Psychopy that many labs use for that.

However, programming visual displays is extremely hardware dependent so for this class we will focus using sound stimuli as sound cards are more consistent across computers.

sounddevice module

In this section we will learn how make use of python to deliver an auditory stimulus using the `sounddevice` module.

To install `sounddevice`, open either a Terminal (mac) or PowerShell prompt from Anaconda Navigator (windows).

Be sure you see the term (base) to the left of the prompt, then type

```
pip install sounddevice
```

`pip` is a package manager for external python modules (those not managed by Anaconda).

In [45]:

```
import numpy as np
from numpy import random
import sounddevice as sd
from matplotlib import pyplot as plt
rng = random.default_rng(seed = 1111)
```

Pure Tones

The simplest auditory stimuli is a pure tone. A pure tone is a sine wave of a particular frequency. It is perceived as having a particular pitch, which you may identify as a musical note.

In [27]:

```

# create a sinusoidal sound wave
sr = 44100      # how many samples per second
                # This is the optimal choice for modern sound cards
time = 0.5      # duration in seconds
volume = 0.5    # DO NOT EXCEED 0.5 EVER

frequency = 440 # cycles per second
time_vec = np.linspace(0, time, int(time*sr)) # A vector of time point starting at 0, ending at time, with sr*time point
test_note = np.sin(frequency * time_vec * 2 * np.pi) # I multiply by 2 pi because the sin function takes radians
# FOR SAFETY. PLEASE LIMIT THE MAXIMUM VOLUME!
test_note = volume*(test_note / np.max(np.abs(test_note)))
# the sample amplitude values must consequently fall in the range of -32768 to 32767.
test_note = 32768*test_note
# they must also have a signed 16-bit integer dtype (=2 bytes) and
test_note = test_note.astype(np.int16) # the astype method changes the data type

sd.play(test_note , sr, blocking=True)
# the parameters here are
# sd.play(sound_array, sample_rate)
# I added, blocking=True to prevent anything else taking command of the sound card, while I am playing my tone.

```

Quick review of sine functions

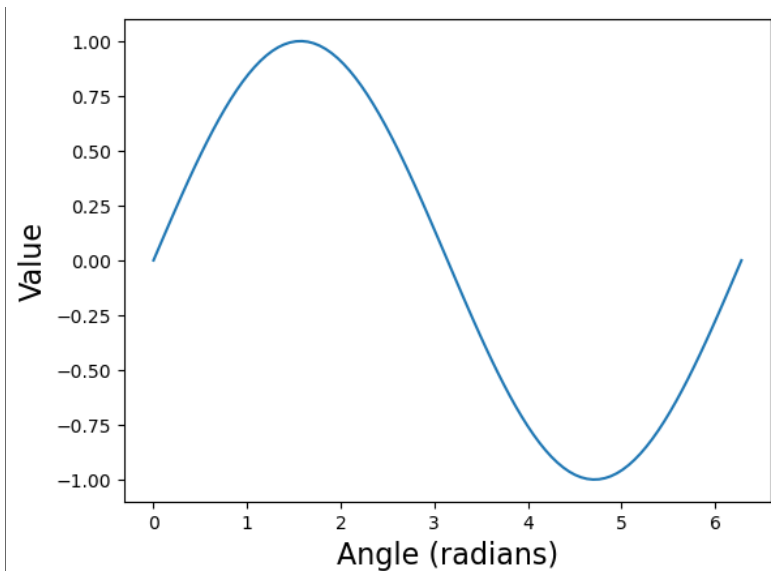
$$x = \sin(\theta) \text{ where } \theta \text{ ranges from } 0 \text{ to } 2\pi$$

In [58]:

```

angle_vec = np.linspace(0, 2*np.pi, 100) # I am using linspace to make 100 samples between 0 and 2*pi
sinefunction = np.sin(angle_vec) # I apply a sin function to theta.
plt.figure()
plt.plot(angle_vec, sinefunction)
plt.xlabel('Angle (radians)', fontsize = 16)
plt.ylabel('Value', fontsize = 16)
plt.show()

```



The concept of frequency

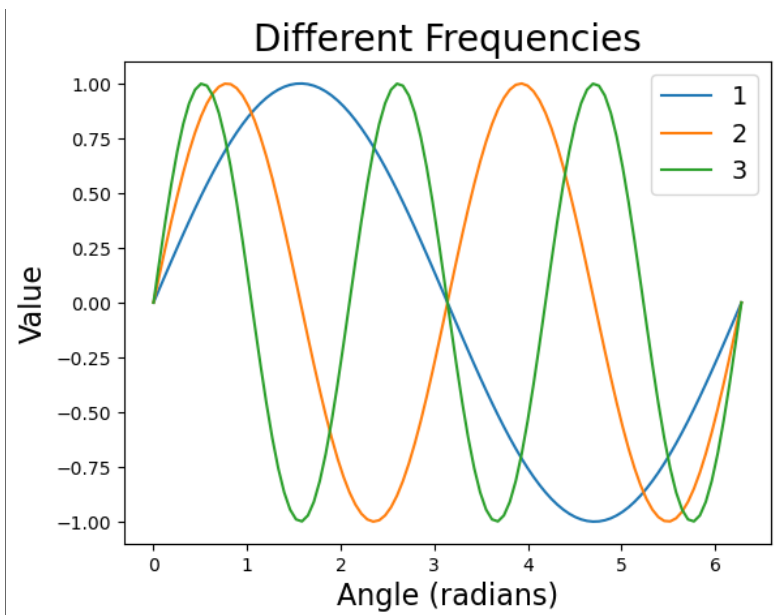
A physicist would tell you frequency is rate of change of phase → how fast do you go around the circle from 0 to 180 (or 2π)

A more intuitive way to say that is that frequency is the number of times the sine function completes a cycle in the period from 0 to 2π

$x = \sin(n\theta)$ where θ ranges from 0 to 2π and n can be any real number

In [59]:

```
angle_vec = np.linspace(0,2*np.pi,100)# I am using linspace to make 100 samples between 0 and 2*pi
plt.figure()
plt.xlabel('Angle (radians)',fontsize = 16)
plt.ylabel('Value',fontsize = 16)
for n in range(1,4,1): # I am looping over 3 values of n
    sinefunction = np.sin((n*angle_vec))
    plt.plot(angle_vec,sinefunction,label = str(n))
plt.legend(fontsize = 14)
plt.title('Different Frequencies',fontsize = 20)
plt.show()
```



Sound unfolds over time.

Sounds are pressure waves unfolding over time.

If we want to make use of our sinusoid as sound stimuli, we need to think about them as a function of time.

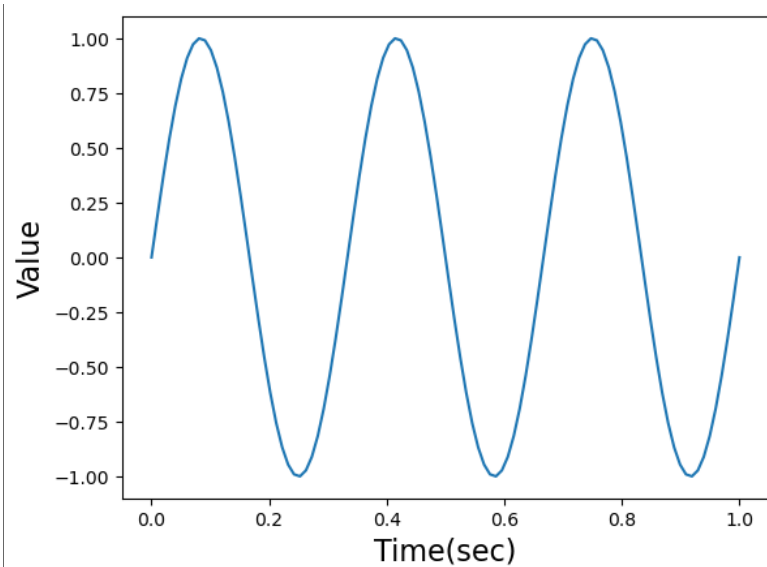
In [30]:

```
duration = 1 # seconds
timesteps = np.linspace(0,duration,100)
anglesteps = timesteps*2*np.pi # This is a mapping between time and angle. In this line of code
                                # I am mapping 1 second of time to the the interval 0 to 2 pi
frequency = 1
sinefunction = np.sin(frequency*anglesteps)
```

- Now, when I plot this, the x-axis is time

In [60]:

```
plt.figure()
plt.plot(timesteps,sinefunction)
plt.xlabel('Time(sec)',fontsize = 16)
plt.ylabel('Value',fontsize = 16)
plt.show()
```



Units in Sound

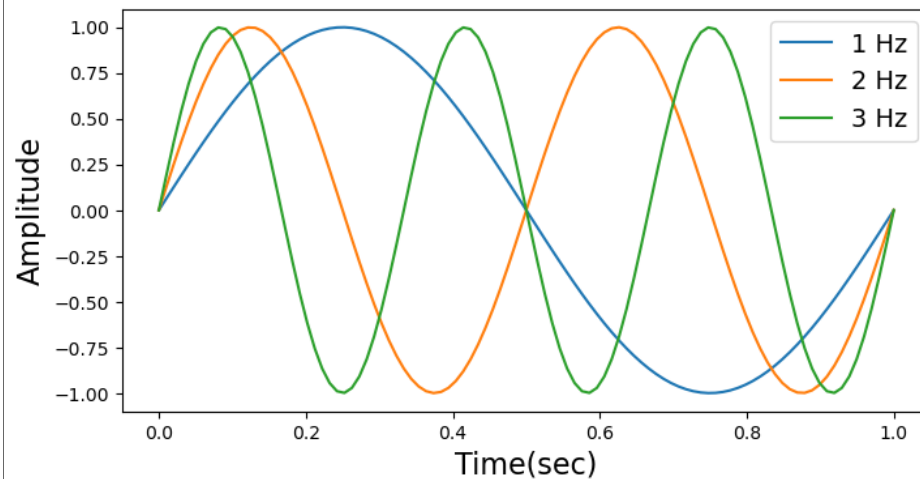
There are several quantities relevant to describing sound

- frequency. The frequency of sound is described in units of Hz or cycles per second.
- Amplitude. The Amplitude of the sound is the size of the wave. The units here (for programming purposes) are arbitrary, since the true amplitude also depends on your sound card, the controls on your computer, etc.

$x = A * \sin(2 \times \pi \times f \times t)$ where f is frequency and A is amplitude and t is time

In [62]:

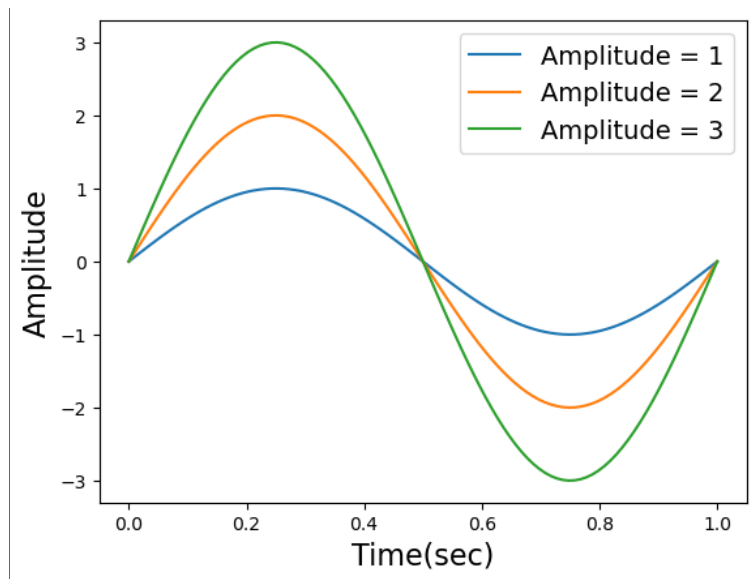
```
#In this example I manipulate frequency
duration = 1 #in seconds
timesteps = np.linspace(0,duration,100)
anglesteps = timesteps*2*np.pi
plt.figure(figsize=(8,4))
for frequency in range(1,4,1):
    sinefunction = np.sin(frequency*anglesteps)
    plt.plot(timesteps,sinefunction,label=str(frequency)+' Hz')
plt.xlabel('Time(sec)',fontsize = 16)
plt.ylabel('Amplitude',fontsize = 16)
plt.legend(fontsize = 14)
plt.show()
```



In [64]:

```
#In this example I manipulate Amplitude
duration = 1
timesteps = np.linspace(0,duration,100)
anglesteps = timesteps*2*np.pi
frequency = 1
plt.figure()
for Amplitude in range(1,4,1):
    sinefunction = Amplitude*np.sin(frequency*anglesteps)
    plt.plot(timesteps,sinefunction,label='Amplitude = ' + str(Amplitude))
```

```
plt.xlabel('Time(sec)', fontsize =16)
plt.ylabel('Amplitude', fontsize =16)
plt.legend(fontsize=14)
plt.show()
```



Sampling Rate

When we make sound with a musical instrument or your voice it is an **analog** signal that is continuous in time.

When specifying sound on a computer, it is a **digital** signal that we have represent it as a discrete set of numbers.

Thus, we have to make a sound with a particular **sampling rate** or number of samples per second. Sampling rate is a frequency that specifies how often we represent a sample of sound (Notice this unit is also Hz).

In the examples above, the sampling rate was implicitly 100 samples/second or 100 Hz in my ``linspace`` command.

The sampling rate you need depends largely on the type of frequencies you need to represent.

Human beings can hear sounds from 20 Hz up to around 20,000 Hz. (really 15,000 Hz, and as you get older that gets lower).

To allow us to play all of these audible sounds, sounds card use a **standard sampling rate of 44,100**.

Sinusoidal pure tone

$$tone = A \sin(2\pi ft)$$

A pure tone has 2 parameters

- A - volume
- f - frequency

And it has to be computed at a set of time points that define its duration.

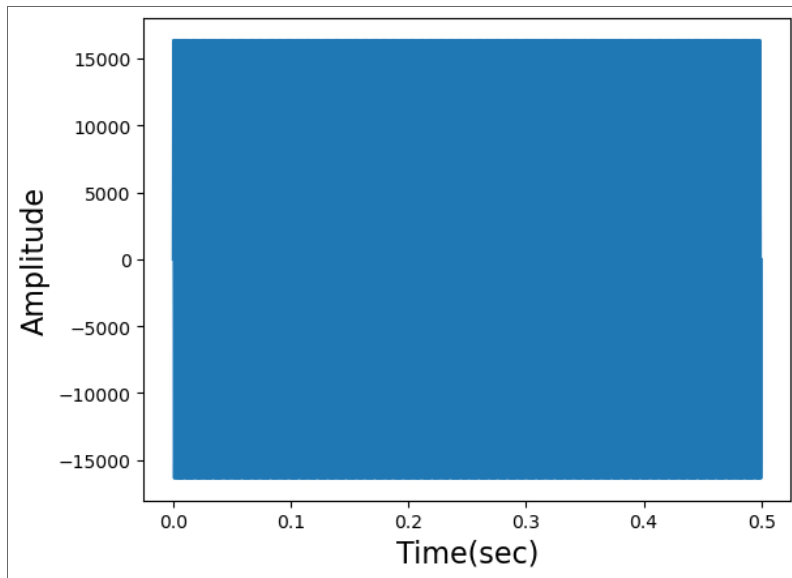
- t - discrete time points

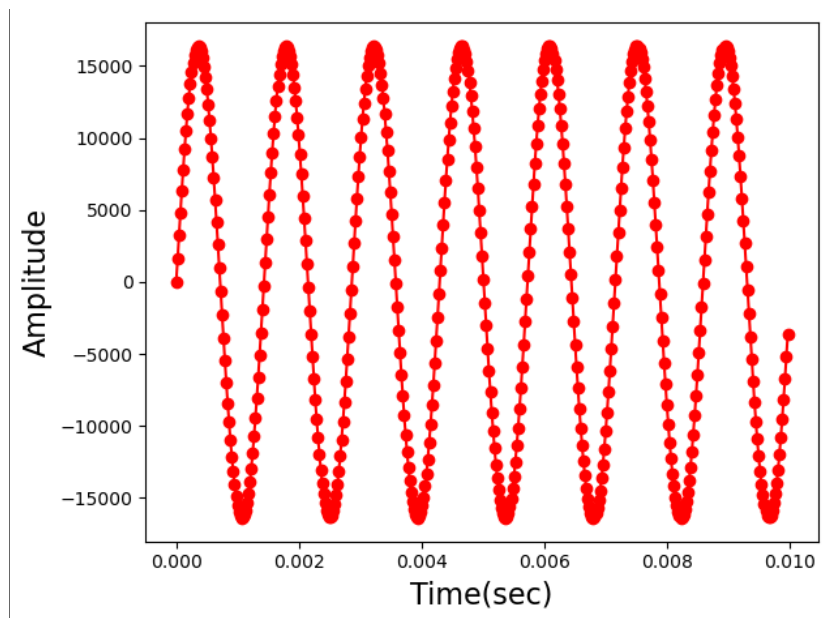
In [34]:

```
# create a sinusoidal sound wave
sr = 44100      # sound card sampling rate
duration = 0.5 # length of the sound in seconds.
frequency = 300 # frequency of the sinewave in Hz
volume = 0.25   # DO NOT MAKE VOLUME LARGER THAN 0.25
time = np.linspace(0, duration, int(duration*sr)) # The number of samples is the length of time X sampling rate.
tone = np.sin(frequency * time * 2 * np.pi) # instead of creating a vector angles, just multiply by 2pi
                                              #inside the call to the sin function.
```

In [72]:

```
plt.figure()
plt.plot(time, tone)
plt.xlabel('Time(sec)', fontsize=16)
plt.ylabel('Amplitude', fontsize=16)
plt.show()
plt.figure()
plt.plot(time_vec[0:441], tone[0:441], 'ro-') #I am just looking at the first 441 samples, which is 0.01 seconds
plt.xlabel('Time(sec)', fontsize=16)
plt.ylabel('Amplitude', fontsize=16)
plt.tight_layout()
plt.show()
```





- ALWAYS MULTIPLY BY VOLUME THAT is 0.25 or less
- When preparing any sound syllabus to play through the sound card, first scale it to -1 to 1 and then apply a volume of 0.25 or less

In [36]:

```
volume = 0.25 # set volume to a safe level
tone = tone/np.max(np.abs(tone)) # this sets the range to -1 to 1
tone = volume*tone # FOR SAFETY. PLEASE LIMIT THE MAXIMIM value of VOLUME to 0.25!
```

A sin function will always return values between -1 and 1
The soundcard expects to get inputs that are 16 bit integers.
16 bit integers fall in the range of -32768 to 32767

```
tone = tone*32767 forces the values to scale so that the
```

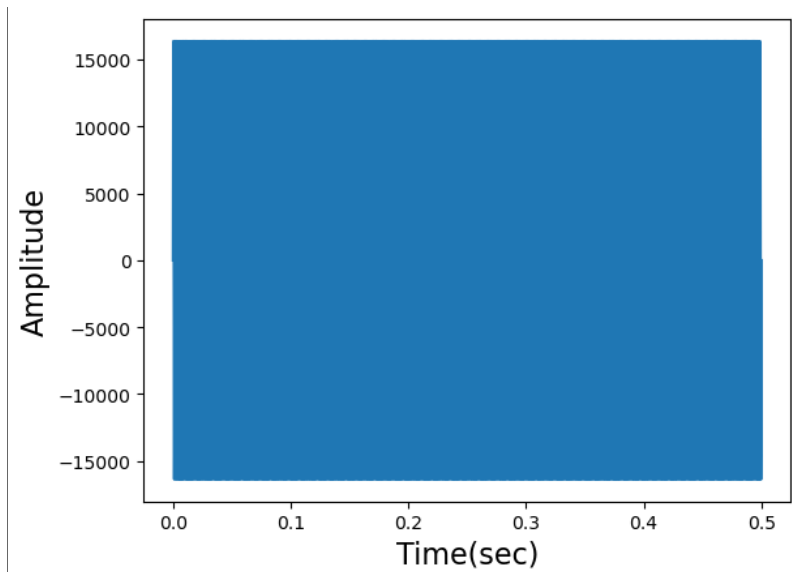
it fills the range from -32767 to 32767

In [37]:

```
tone = tone*32767 # scale to the range of the sound card.
tone = tone.astype(np.int16) #the astype method changes the data type
```

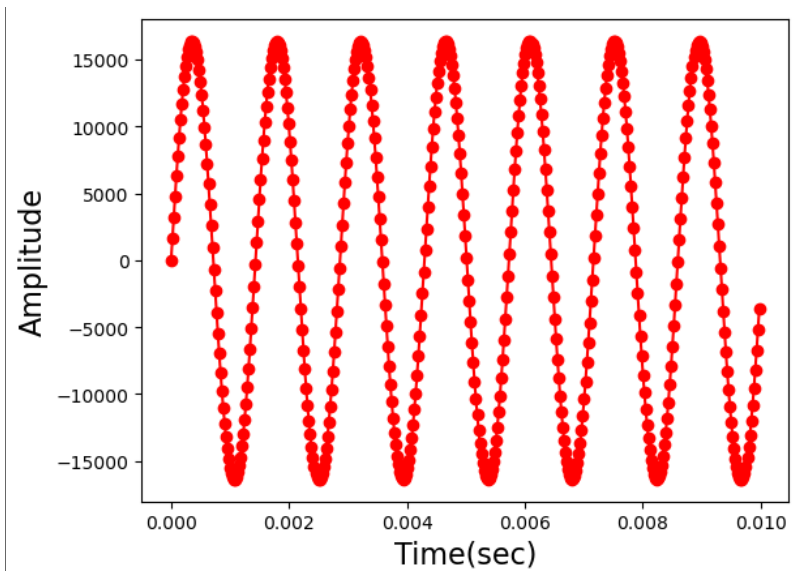
In [73]:

```
plt.figure()
plt.plot(time,tone)
plt.xlabel('Time(sec)',fontsize=16)
plt.ylabel('Amplitude',fontsize=16)
plt.show()
```



In []:

```
plt.figure()
plt.plot(time_vec[0:441],tone[0:441], 'ro-')
plt.xlabel('Time(sec)',fontsize=16)
plt.ylabel('Amplitude',fontsize=16)
plt.show()
```



In [40]:

```
sd.play(test_note, sr, blocking=True)
#there are 4 required paramters here.
# audio_data - the numpy array or wav object you want to play
# sample_rate= sampling rate, or samples per second, default to 44100 (its good practice to specify this)
# blocking= True/False. If True, the code will wait until the sound is done playing before moving on.
```

Musical notes correspond to different frequencies of sound.

A can have frequencies 440, 880, 1760, 3520,

These frequencies are the start of an octave that contains 12 different notes.

For example:

- A = 440
- Bb = 466 (B flat)
- B = 494
- C = 523
- C# = 554 (C sharp)

- D = 587
 - Eb = 622 (E flat)
 - E = 659
 - F = 698
 - F# = 740 (F sharp)
 - G = 784
 - Ab = 831 (A flat)
-

- A = 880
- There are a number of ways to organize this information.
- This is ugly

In [41]:

```
notes = ['A', 'Bb', 'B', 'C', 'C#', 'D', 'Eb', 'F', 'F#', 'G', 'Ab']  
freqs = [440, 466, 494, 523, 554, 587, 622, 659, 698, 740, 784, 831]
```

- Why is this useful?
- Why is this not useful?

Dictionaries

Dictionaries are a native Python data type that are useful to organize information.

They can be used to organize any type of variables, including variables, lists, arrays, matrices, and mix types together. (They are my favorite way to organize information)

At the core of a dictionary is **key value** pairing.

In [42]:

```

prof = dict()
#keys can be strings
prof['First'] = 'Ramesh'
prof['Last'] = 'Srinivasan'
#values can be numeric
prof['age'] = 58
#keys can have spaces
prof['years at UCI'] = 25
prof['undergrad'] = ['UPenn']
prof['gradschool'] = ['Tulane']
#values can be arrays
prof['dob'] = np.array([1,21,1967])
#keys can be numeric
prof['jobs'] = ['Oregon', 'Scripps', 'UCI']
prof['city'] = {'Mumbai':2, 'Manila':15, 'Philadelphia':4, 'New Orleans':6, 'Eugene':3, 'San Diego':3, 'Irvine':25}

```

In [43]:

```

print(prof.keys())
print(prof['city'].keys())

```

```

dict_keys(['First', 'Last', 'age', 'years at UCI', 'undergrad', 'gradschool', 'dob', 'jobs', 'city'])
dict_keys(['Mumbai', 'Manila', 'Philadelphia', 'New Orleans', 'Eugene', 'San Diego', 'Irvine'])

```

In [44]:

```

dob = prof['dob']
print(dob)

```

```
[ 1  21 1967]
```

- Dictionaries are useful when we have various types of information we want to hold together in a container.
- Here I make a dictionary with the information about musical notes

In [46]:

```

notes = dict()
notes = {'A':440, 'Bb':466, 'B':494, 'C':523, 'C#':554, 'D':587, 'Eb':622, 'F':659, 'F#':698, 'G':740, 'Ab':831}
#I hate the above syntax, but i see why people do it. i prefer
#notes['A'] = 440
#notes['Bb'] = 466

```

```
#....
```

In [51]:

```
notes['names'] = ['A', 'Bb', 'B', 'C', 'C#', 'D', 'Eb', 'F', 'F#', 'G', 'Ab']
notes['frequencies'] = [440, 466, 494, 523, 554, 587, 622, 659, 698, 740, 831]
print(notes['names'])
print(notes['frequencies'])
```

```
['A', 'Bb', 'B', 'C', 'C#', 'D', 'Eb', 'F', 'F#', 'G', 'Ab']
[440, 466, 494, 523, 554, 587, 622, 659, 698, 740, 831]
```

In [56]:

```
# play note F# from dictionary
sr = 44100 # sound card sampling rate
duration = 0.5 # length of the sound in seconds.
frequency = notes['F#'] # frequency of the sinewave in Hz
volume = 0.5 # DO NOT MAKE VOLUME LARGER THAN 0.25
time = np.linspace(0, duration, int(duration*sr)) # The number of samples is the length of time X sampling rate.
tone = np.sin(frequency * time * 2 * np.pi) # instead of creating a vector angles, just multiply by 2pi
```

In [57]:

```
tone = tone / np.max(np.abs(tone)) # scale to the range of the sound card.
tone = volume*tone # FOR SAFETY. PLEASE LIMIT THE MAXIMIM value of VOLUME to 0.5!
tone = tone*32767 # scale to the range of the sound card.
tone = tone.astype(np.int16) # always convert to 16 bit integers
sd.play(tone, sr, blocking=True)
```

In []: